Cognizant

COGNIZANT

Cognizant Academy

Medical Laboratory

C#, ADO .Net Core Knock Out

ChallengeVersion 1.0

	Prepared By / Last Updated By	Reviewed By	Approved By
Name	Gnanasekar R		
Role			
Signature			
Date			

Cognizant

Table of Contents

1.0 Introduction **1**

1.1 Purpose of this document **1**

1.2 Definitions & Acronyms **1**

1.3 Project Overview **1**

1.4 Scope **2**

1.5 Target Audience **2**

1.6 Software Requirement **2**

1.6.1 Software Requirements 2

2.0 Functional Requirements **2**

2.1 Functional Requirements **2**

2.2 Use case Diagram **3**

2.3 System Architecture Diagram **4**

2.4 Sample Input/Output **5**

3.0 Design Specification **6**

3.1 Data Design **6**

3.2 Component Details for identified Use Cases **7**

3.2.1 Get data and insert into the database. 7

3.2.2 Get medicineId and update Details from database. 8

3.2.3 Calculate Total Price 8

3.3 Component Specification **8**

3.3.1 UpdateDeliveryStatus Method 9

3.3.2 RetrieveAllDetails Method 10

3.3.3 CalculateTotalPrice Method 10

3.3.4 GetConnection Method 10

3.4 General Design Constraints **11**

4.0 Submission **11**

4.1 Code submission instructions **11**

5.0 Change Log **11**

6.0 Evaluation Areas **12**

1.0 Introduction

1.1 Purpose of this document

A medical Laboratory maintains a medicine log of their New 10 Laboratories.

This medicine log contains the various medicines that are available in the company and the status of the medicines.

Their current focus is to automate the below mentioned requirements.

1. Retrieve all the medicines details from the database.

2. Update the delivery status of the medicines for the given medicine id in the database.

3. Calculate medicine discount and total price.

1.2 Definitions & Acronyms

Definition / Acronym	**Description**
Req	Requirement

1.3 Project Overview

This project captures the various concepts, techniques and skills learnt and helps to put them into practice using C# with ADO.NET. Admittedly, this would be at a scaled-down level since the purpose is to let the associate experience the various concepts learned in C# as an individual. The individual associate is expected to create a console-based application within the specified time.

1.4 Scope

The scope of the system is explained through its following modules

1. Retrieve all the medicines details from the database.

2. Update the delivery status of the medicines for the given medicine id in the database.

3. Calculate medicine discount and total price.

1.5 Target Audience

Learner Level

1.6 Software Requirement

1.6.1 Software Requirements

#	Item	Specification/Version
1.	C#	10
2.	ADO.NET	6.0
3	SQLSERVER	2017

Note: All the required hardware and software will be provided in the Tekstac platform.

2.0 Functional Requirements

2.1 Functional Requirements

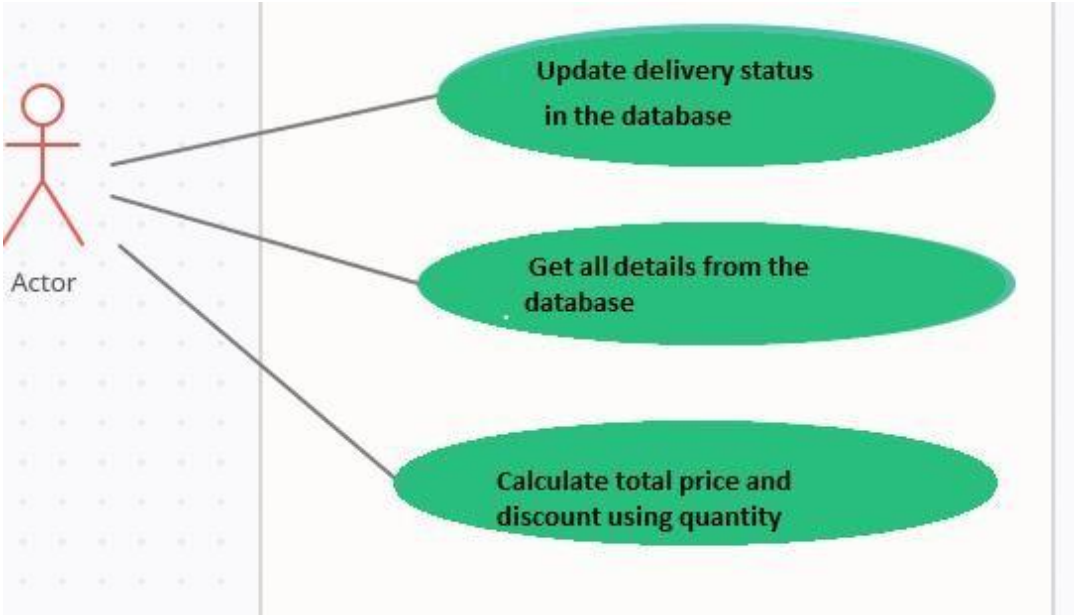
Req. #	1
Req. Name	Get the medicine id and delivery status from the user. Update the details to the database.
Req.	The medicine id and delivery status are entered

Description	through the Main method. Update the delivery status in the Medicine table in the database.
Actors/ Users	Admin
Comments	The admin is responsible for updating the data .

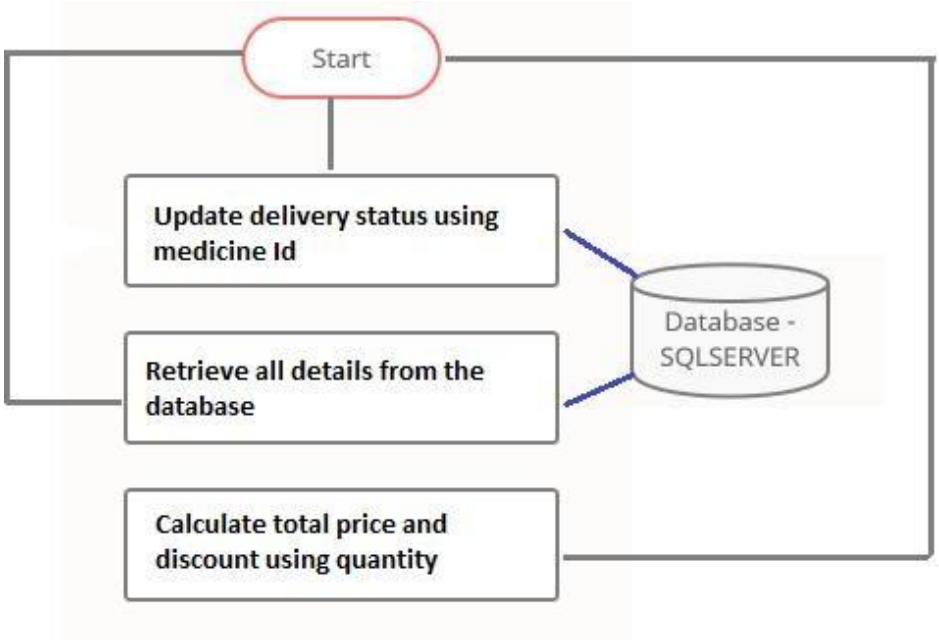
Req. #	2
Req. Name	Retrieve all the data present in the database
Req. Description	Retrieve all the data in the database and store the data in the list class.
Actors/ Users	Admin
Comments	The admin is responsible for retrieving the data from the database

Req. #	3
Req. Name	Calculate total price charges based on quantity.
Req. Description	The user enters the quantity. It will calculate the total charges and discount based on the quantity.
Actors/ Users	User
Comments	The user is responsible for entering the quantity.

2.2 Use case Diagram



2.3 System Architecture Diagram



2.4 Sample Input/Output

```
1. Update the delivery status
2. Retrieve all details
3. Calculate total price
4. Exit
Enter your choice
1
Enter the medicine id
119
Enter the delivery status
Delivered
119      Aorus         Bala         120         1900       Delivered
```

```
1. Update the delivery status
2. Retrieve all details
3. Calculate total price
4. Exit
Enter your choice
2
119        Aorus       Bala       120       1900         Delivered
120        prime       Subham       100       1200         Not delivered
121        Tomhawk       senthil       200       2800         not delievered
```

```
1. Update the delivery status
2. Retrieve all details
3. Calculate total price
4. Exit
Enter your choice
3
Enter the quantity
250
11700
```

```
1. Update the delivery status
2. Retrieve all details
3. Calculate total price
4. Exit
Enter your choice
3
Enter the quantity
20
1040
```

3.0 Design Specification

3.1 Data Design

Table Structure:

Table name: Medicine

Column Name	**Data type**
MedicineId	int(Primary Key)
MedicineName	varchar
PharmacyName	varchar
Quantity	int
TotalPrice	Float precision
DeliveryStatus	varchar

Design Constraints:

- Use **SQLSERVER** database to store the data. The database name is "dbTraineeHandsOn". **This is already created for you in Tekstac.**
- The table names and the column names should be the same as specified in the table structure.
- The database connection information is specified in the "appsettings.json" file, which is also provided as part of code skeleton. THIS IS GIVEN ONLY FOR YOUR REFERENCE. You need NOT change this.

Sample data present in the **Medicine** database

MedicineId	MedicineName	PharmacyName	Quantity	TotalPrice	DeliveryStatus
119	Aorus	Bala	120	1900	Delivered
120	Prime	Subham	100	1200	Not delivered
121	Tomhawak	Senthil	200	2800	Not delivered

Note: The code skeleton will be available in the Tekstac platform

3.2 Component Details for identified Use Cases

3.2.1 Get data and insert into the database.

In the 'Main' get the following values,

MedicineId
MedicineName
PharmacyName
Quantity
TotalPrice
DeliveryStatus

3.2.2 Get medicineId and update Details from database.

The User enters the medicineId then it should update delivery status of the medicine that matches the medicineId.

Example: If medicineId is 1 and then it should update the information of DeliveryStatus.

3.2.3 Calculate Total Price

The User enters quantity then, the method should calculate the total charges and return it.

If quantity is between 50 to 99(inclusive) then the discount is 5% and charge of the medicine is INR 52.

If quantity is between 100 to 199(inclusive) then the discount is 7% and charge of the medicine is INR 52.

If quantity is between 200 to 499(inclusive) then the discount is 10% and charge of the medicine is INR 52.

If quantity is greater than or equal to 500 then the discount is 10% and charge of the medicine is INR 52.

If quantity is less than 50 then the no discount

Example: quantity = 100

price = 100 * 52

Discount is 7% off from the price

Total Charge = 4836

3.3 Component Specification

Class Name : MedicineDetails(model class)

Responsibility:

This model object holds the state of the medicine detail at all point-in-time.

Type(Class)	Properties
MedicineDetails	**int** MedicineId **string** MedicineName **string** PharmacyName **int** Quantity **double** TotalPrice **string** DeliveryStatus

Note: Keep all the properties '**public**'.

Class Name : MedicineUtility (utility class)

3.3.1 UpdateDeliveryStatus Method

Type(Class)	Method	Responsibilities
MedicineUtility	public MedicineDetails UpdateDeliveryStatus (int medicineId, string deliveryStatus)	This method should accept an medicine id and delivery status then execute an sql query to update the details into the database then return the update details and store the details in **MedicineDetails** object and return that object.

3.3.2 RetrieveAllDetails Method

Type(Class)	Method	Responsibilities
MedicineUtility	Public List<MedicineDetails> RetriveAllDetails ()	This method should retrieve all the medicine details of the medicine that is in the database. Retrieve all the details and store it in a List<MedicineDetails> and return it.

3.3.3 CalculateTotalPrice Method

Type(Class)	Method	Responsibilities
MedicineUtility	public double CalulateTotalPrice (int quantity)	This method is used to calculate the totalprice based on the quantity. calculate the totalprice and return it as data type double. Refer 3.2.3 section for the charges

Class Name : DBConnection (DAO class)

3.3.4 GetConnection Method

Responsibility:

The below given method is already provided. Your task is to return

the SqlConnection String from this method.

Type(Class)	Method	Resources
DBConnection	public SqlConnection GetConnection()	This method is used to connect to SQLSERVER database. Use the connection name "DBConnection" in appsettings.json to create a connection.

3.4 General Design Constraints

1. The properties/method/class name should be correctly specified as given in the document.
2. Keep all the classes as '**public**'
3. Do not change the **appsettings.json** file.
4. Do not change the namespace name.

4.0 Submission

4.1 Code submission instructions

1. Do not change the code skeleton given as your code will be auto evaluated.
2. You can validate your solution against sample test cases during the assessment duration.
3. Your last submitted solution will be considered for detailed evaluation.
4. Make sure to submit the solution before the time limit. After the assessment duration you will not be allowed to submit the solution.

5.0 Change Log

	Changes Made
V1.0.0	Initial baseline created on <dd-Mon-yy> by <Name of Author>
Vx.y.z	<Please refer the configuration control tool / change item status form if the details of changes are maintained separately. If not, the template given below needs to be followed>

	Section No.	Changed By	Effective Date	Changes Effected

6.0 Evaluation Areas

S.No	Description
1.	Declaration of properties in the class MedicineDetails
2.	Declaration of methods in the class DBConnection and MedicineUtility
3.	Implementation to create a valid database connection object
4.	Implementation to Update delivery status details to database
5.	Implementation to Get all medicine details from database
6.	Calculate the total price based on the given business rules.